

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Ragavi V	Reg. No.:	192421192
Department:	B.TECH(IT)	Date:	2/08/2026

1. Signed Integer Addition Using 2's Complement

Problem

An 8-bit signed integer addition produces incorrect results when adding **+120** and **+50**.

Binary Representation

$$+120 = 01111000$$

$$+50 = 00110010$$

$$\text{Result} = 10101010$$

The binary result is **10101010**, which represents **-86** in 2's complement.

Debugging

- Expected decimal result:

$$120 + 50 = 170$$

- An 8-bit signed integer can represent only:

-128 to +127

- Since **170 > 127**, it cannot be represented.

Overflow Detection

Overflow occurs because

- Both operands are **positive**
- Result becomes **negative**

Or using carry bits:

Carry into MSB $\neq$ Carry out of MSB

Hence,

Overflow = 1

Solution

- Check overflow flag after every addition.
- If overflow occurs:
 - Display "Overflow Error"
 - Use a larger data type (16-bit, 32-bit, or 64-bit).

Conclusion

Overflow occurs because the result exceeds the representable range of an 8-bit signed number.

2. Ripple Carry Adder Performance Bottleneck

Problem

Arithmetic operations are slow because the processor uses a Ripple Carry Adder (RCA).

Debugging

In Ripple Carry Adder,

Carry₀ → Carry₁ → Carry₂ → Carry₃ → ... → Carry₇

Each bit must wait for the previous carry.

Example:

$$A = 10110110$$

$$B = 01001011$$

Carry propagates through every stage.

Delay

$$T = n \times \text{Carry Delay}$$

For 8 bits,

$$\text{Delay} = 8 \times T_c$$

Performance Issue

- Sequential carry propagation
- Large delay
- Not suitable for real-time systems

Optimized Solution

Use **Carry Look-Ahead Adder (CLA)**.

CLA computes carries using

Generate (G)

Propagate (P)

Where

$$G_i = A_i B_i$$

$$P_i = A_i \oplus B_i$$

Carry equations

$$C1 = G0 + P0C0$$

$$C2 = G1 + P1G0 + P1P0C0$$

Carries are generated simultaneously.

Advantages

- Much faster
- Parallel carry generation
- Reduced propagation delay
- Better real-time performance

Conclusion

Replacing RCA with CLA greatly reduces arithmetic execution time.

3. Booth's Multiplication Debugging

Problem

Booth's algorithm produces incorrect results for negative operands.

Debugging Steps

Booth checks

(Q0,Q-1)

Rules

00 → No operation

01 → $A = A + M$

10 $\rightarrow A = A - M$

11 $\rightarrow$ No operation

After each operation

- Perform arithmetic right shift.
- Preserve sign bit.

Possible Errors

1. Incorrect bit-pair checking
2. Logical shift used instead of arithmetic shift
3. Missing sign extension
4. Wrong subtraction operation
5. Incorrect initialization of Q-1

Example

$M = -5$

$Q = 3$

If logical shift is used,

Sign bit becomes 0

Negative value changes into a positive value.

Final product becomes incorrect.

Corrections

- Use arithmetic right shift.
- Maintain sign extension.

- Correctly initialize

$Q-1 = 0$

- Follow Booth decision table correctly.

Conclusion

Proper sign extension and arithmetic shifting ensure correct signed multiplication.

4. Restoring Division Optimization

Problem

Restoring division performs unnecessary restoration operations, increasing execution time.

Debugging

Workflow

Shift

Subtract divisor

If result < 0

↓

Restore previous value

↓

Continue

Every negative subtraction performs

Restore + Next Iteration

This increases computation.

Performance Issue

- Extra addition
- More clock cycles
- Increased execution time

Optimized Solution

Use **Non-Restoring Division**.

Instead of restoring,

If negative

↓

Next iteration performs addition

Else

↓

Perform subtraction

No restoration after every failed subtraction.

Advantages

- Fewer arithmetic operations

- Reduced execution time
- Higher processor efficiency

Conclusion

Non-restoring division eliminates repeated restoration steps, making division faster.

5. IEEE 754 Floating-Point Accuracy

Problem

Adding a very small number to a very large number gives inaccurate results.

Example

$100000000 + 0.000001$

Debugging

IEEE 754 addition performs

1. Exponent alignment
2. Mantissa addition
3. Normalization
4. Rounding

Example

Large exponent

↓

Small number shifted right many times



Small value lost

This causes

Loss of Precision

Sources of Error

- Exponent alignment
- Mantissa truncation
- Rounding
- Limited precision (23-bit fraction)

Optimized Solutions

1. Use **Double Precision (64-bit)**
 - 52-bit mantissa
 - Higher accuracy
2. Use **Kahan Summation Algorithm**
 - Reduces accumulated rounding errors
3. Add numbers from **smallest to largest**
 - Improves numerical stability
4. Avoid repeated subtraction/addition of vastly different magnitudes

Conclusion

Using double precision and improved summation algorithms significantly reduces floating-point errors.

Code of Conduct:

I V.Ragavi/192421192(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: